UNIT: a type checker for dependent type theory

Thierry Coquand

Abstract

We present UNIT, a type checker for dependent type theory with a module system in the tradition of AUTOMATH and a uniform treatment of definitions. There is no unification, no instance search and no tactics, and there is one uniform unfolding policy, without priorities for particular definitions or choices adapted to the problem, so the work of checking can be observed and compared with the informal proof. With UNIT we have checked a model of univalent type theory, synthetic algebraic geometry up to Châtelet's theorem, synthetic Stone duality with its cohomology, and results of commutative algebra, among them finite free resolutions and Wiebe's theorem. These developments show that a small, uniform system supports substantial mathematics, and give concrete reasons to reconsider the complexity routinely accepted in proof assistants.

1 Introduction

Large proofs are now checked in dependent type theory. Its basic ideas go back to AUTOMATH: propositions as types, proofs as terms checked in a context, and definitions that are not expanded in the proof but expressed by convertibility. That definitions belong to the calculus, and are unfolded by a reduction of their own, is the point of de Bruijn's $\Lambda\Delta$ [16], where a definition is a declaration of the same kind as an abstraction and can occur anywhere in a context. Here we investigate a particular combination: shared representations of definitions, a uniform conversion strategy, and reports of the definitions unfolded, compared with an accompanying mathematical proof.

So far, proof systems have been designed for interaction with a human user, and pure dependent type theory has been extended for this purpose: for instance with unification and implicit arguments, type classes or instance search, coercions, tactics, impredicative propositions and definitional proof irrelevance. Each serves a purpose in large-scale formalisation, but they make the work of checking harder to relate to the proof being checked, which is what we want to study here, and they are not necessary for representing mathematical arguments: the developments of Section 4 were written without unification-based argument inference, instance search, coercions, tactics or a proof-irrelevant sort; omitted arguments are only filled by one-way matching (Section 2.4). We therefore work with a small dependent type theory with definitions, modules and records, in which nothing is guessed and every unfolding of a definition can be observed. In particular there are no tactics: a proof is the term that is checked; and there are no placeholders to be refined interactively: a file is checked as it is written.

This paper presents such a system, UNIT (Section 2); the checker and all the developments are available at `https://github.com/coquand/unit`. Its type theory has Π, Σ, records, data types with primitive recursion, an identity type, and cumulative universes indexed by level expressions. Its module system follows the segments of AUTOMATH: a theorem is stated in a context, and to use it one gives an instance of the context, the objects and the proofs of the hypotheses, in order; an instance only binds the parameters, and the body is neither copied nor checked again. Its treatment of definitions is uniform: a definition is a declaration like the others, it keeps one identity wherever it is used, and conversion tries to recognise two values by their names before it opens anything, and then opens one definition at a time.

This uniformity is not only a matter of simplicity. In [15] the author recalls that definitions are used in type theory differently from the way Frege used them in the *Begriffsschrift* [21], and that, on the first lemmas of the *Begriffsschrift* translated into AUTOMATH notation, type checking felt similar to what one does when trying to understand a proof. We ask whether this holds beyond this example. We consider detailed, precise informal proofs that identify the named constructions and lemmas used and explain the necessary openings of definitions. Ordinary texts often leave such openings silent; making them explicit is part of the comparison, not an assumption about all mathematical reading. We make two hypotheses about these selected aspects of checking.

(H1) *Checking can follow reading.* On a proof accompanied by such a text, a checker can recognise repeated uses of named constructions without unfolding them, and its necessary openings of definitions can be accounted for by the text or by a stated representation convention. This concerns recognition and unfolding; substitution, universe constraints, scope checking and argument recovery remain additional work of the checker.

(H2) *Cost is a diagnostic.* When a step that is routine on paper is expensive to check, the report can help locate repeated decoding of a representation, a missing interface or intermediate statement, or unnecessary work in the checker. This is a heuristic, not an exhaustive diagnosis or a complexity bound: some proofs do compute.

For (H2) to be usable the checker must not hide the cost: a checker with clever unfolding heuristics can make a badly structured proof cheap, and so lose the information. A uniform checker makes its work observable. This should be relevant to the problem of proofs *lost in translation* [5]: a formal proof can be accepted without being the proof of the text it claims to translate. Unfolding reports supply evidence for investigating this correspondence. An unexplained opening may be representational work within the same mathematical argument; agreement of reports cannot exclude a different argument or an incorrectly translated statement.

The method (Section 3) uses this. It has three parts. (1) The checker reports, for each declaration, how its comparisons were settled and which definitions it opened. (2) The informal proof is written, and each step is classified by what a reader does: match names, open a definition the text cites, or nothing. The definitions the text cites can be given to the checker, which then reports the others. (3) For a declaration that costs more than its text suggests, the formal proof is split into the steps of the informal proof, each step is measured, and the expensive comparisons are traced to the part of the checker that asks for them. The method can be applied to an existing development, or while writing a new one.

The aim (Section 5) is that the structure of a formal proof be that of a detailed, precise informal proof: one definition for each notion, one declaration for each step, the intermediate claims written, "by definition" where a definition is opened, and each fact that a text states together with a definition proved once, as a lemma that holds by definition. Acceptance does not establish that a formal proof follows a given text [5]; the method supplies local evidence for examining this question.

The claim. UNIT demonstrates that a small, uniform system supports substantial mathematics. Its developments, more than thirty thousand declarations, go from a model of univalent type theory to Châtelet's theorem, to the comparison of sheaf and Čech cohomology of compact Hausdorff spaces, and to finite free resolutions in commutative algebra, with no unification, no instance search, no tactics, and one uniform unfolding policy without definition-specific priorities or adaptive choices, and each of them is checked in under a minute (Table 1). Each notion

these systems add solves a real problem; the developments show that the problem can also be solved by writing, with named instances, explicit structures and stated lemmas, and in the cases we examined (Section 4) this brought the formal proof closer to its text. This is a concrete reason to reconsider the complexity routinely accepted in proof assistants for mathematics.

AI-assisted formalisation makes the question pressing. Much of that complexity exists to spare a human author from writing what can be inferred. When the formal text is written by a machine, explicit arguments still cost something to generate, maintain and review, but sparing a human the typing is no longer a decisive reason for complexity in the proof language or in the checker; what matters is that the result can be checked simply and read against the mathematics. All the developments reported here were written in this way. Existing proof assistants remain valuable systems and libraries, but their architecture need not be the inevitable model for what comes next: it is a reason to reconsider which complexities are necessary in the proof language and in the checker.

Results (Section 4). Table 2 gives what checking opens in each development.

2 The checker

2.1 Units and segments

A *unit* is a declaration:

$x\,(\Delta) : A$	a parameter
$x\,(\Delta) : A = t$	a definition
$\texttt{data}\ D\,(\Delta) = c_1(\Delta_1) \mid \ldots$	a data type
$\texttt{level}\ a, \quad \texttt{constraint}\ \psi$	levels and constraints
$\texttt{module}\ M\,(\Delta)\,\{\,\vec{u}\,\}, \quad \texttt{record}\ R\,(\Delta)\,\{\,\vec{u}\,\}$	a segment
$\texttt{module}\ N = M[\vec{a}]$	an instance
$\texttt{opaque}\ f, \quad \texttt{transparent}\ f$	the unfolding strategy

Terms are those of a dependent type theory with Π, Σ, an identity type with its eliminator J, labelled sums with their eliminators (`split`), records, cumulative universes $\texttt{U}\,l$ indexed by level expressions, and products over levels and over constraints between levels [3].

A file, a module body, a record body and a local block are all lists of units, checked in the same way: each unit in the context of the units before it, and then added to it; a definition stays a definition, with its body available for computation. The scope is sequential: a unit sees only the units before it, never itself or a later one. This is the block structure of Automath, and, as in $\Lambda\Delta$ [16], a context contains definitions as well as declarations, in any order. The context after a unit does not depend on whether the unit checked: a definition enters the context with its type and its body, not with its proof of correctness. So each declaration can be checked separately, with its own time budget, and one file can be checked while the others are trusted, as a reader accepts a lemma and reads on. This serves diagnosis only: a development is accepted when all its declarations check, and the success of a declaration is relative to the context before it, so that after a failure it is conditional on the failed declaration.

A *segment* is a named list of units; its parameters are its units without a body. It is checked once, with its parameters unknown. An instance $M[\vec{a}]$ checks each a_i against the type of the ith parameter with the earlier ones substituted; nothing of the body is checked again. A module and a record are the same object and differ only in what is given: an instance gives every parameter; a record type $R[\vec{p}]$ gives those of the header, and $\texttt{new}\,R[\vec{p}, \vec{v}]$ gives the others, its fields. Fixing a field, $R[\vec{p}][x := e]$, gives the type of the elements whose field x is e. A hypothesis can be a parameter of a whole development and be discharged by one instance: the

model of univalent type theory below assumes function extensionality and univalence as two parameters, and one instance, which checks two arguments, gives it under univalence alone.

Data types are strictly positive. There is no recursion by name: a function out of a data type is a `split`, and a branch names, after its pattern variables, the recursive results, which are hypotheses like any other:

```
data N = zero | suc (n : N) ;
plus (n : N) : N -> N = split { zero -> n ; suc m | r -> suc r } ;
```

This is primitive recursion at higher types, as in Gödel's system T, written as the inductive case expressions of Burstall [6], where termination follows from the scope rules.

2.2 Structures without instance search

Proof assistants commonly find structure arguments by instance synthesis, or solve unification problems on record projections by canonical structures; both choose automatically an argument that could be given explicitly. Unit has neither. A mathematical structure is a record, generic mathematics about it is a module whose parameter is an element of the record, and `open` gives the operations and laws short names:

```
record CRing {
  R : U a ;  z : R ;  o : R ;
  add : R -> R -> R ;  mul : R -> R -> R ;  neg : R -> R ;
  addC (x y : R) : Id R (add x y) (add y x) ;
  addZ (x : R) : Id R (add x z) x ;  ...
} ;
module RL (Rg : CRing) {
  open Rg ;
  unit (x : R) : U a = (y : R) * Id R (mul x y) o ;
  zl (x : R) : Id R (add z x) x = trans (addC z x) (addZ x) ;
  ...
} ;
```

The ring is given once, for a whole development, as a text says "let R be a commutative ring" and then uses its operations; an instance `RL[Rg]` makes all the results available for a particular ring. The choice is explicit, but it is made once, not at each step.

The difficulty that instance search addresses arises when a proof moves among many related structures: a ring, its additive group, a module over it, a ring of polynomials. With explicit structures each passage is a named construction, and the same structure reached by two constructions must be identified explicitly. In the cohomology of synthetic Stone duality, the groups of cocycles are both kernels of homomorphisms and groups of the Čech complex; statements written with one name and used with the other showed in the report as repeated openings (one declaration caused 3708 unfoldings), and stating the types in the terms of one construction removed them (447). In the developments of Section 4, records, parameterised modules and `open` have sufficed to organise the structures without instance search, and supplying structure arguments at the level of a development has introduced little repetition.

2.3 Definitions as closures

Evaluation maps terms to values in an environment. Following $\Lambda\Delta$, a definition is not replaced by its body when it is declared; it is unfolded only where its body is needed. The use of closures comes from [8, 30], and was used in the implementation of `cubicaltt` [9], which also introduced `opaque` and `transparent`. A use of a definition f evaluates to the closure $\langle f; \rho \rangle$ of

the unit and the environment ρ where it is declared. The *arguments* of a use are the values in ρ of all the variables and levels in scope at the declaration. A member $M[\vec{a}].f$ of an instance is $\langle f; \rho, \vec{a} \rangle$. Nothing is copied: f has one identity wherever it is used.

`opaque f` declares that from this point on f is not unfolded, and `transparent f` undoes it. The strategy is a parameter of the run: evaluation builds the same values whatever the strategy. Making a definition opaque can only make fewer terms convertible, so an `opaque` declaration never makes an incorrect proof correct. Its use is the mathematical one: once a construction is characterised by lemmas, the rest of the development uses the lemmas.

2.4 Conversion

Definitional equality is generated by β (for functions, pairs, levels, projections, `split` on a constructor and J on reflexivity), the unfolding of definitions, and η for functions and pairs. A data type and a `split` are constants, as a definition is in $\Lambda\Delta$: a data type is applied to its parameters, and a `split` is lifted over the whole context in which it is written, as local pattern-matching functions usually are. A `split` computes only on a constructor, and two occurrences of the same constant are equal when their arguments are, an argument that a branch does not use being still an argument. This is part of the declarative equality, not a restriction of the algorithm: the algorithm is meant to decide exactly this equality, although we give no proof that it does (Section 7). Two values are compared as follows.

1. *By names.* They are equal if they are strictly equal, that is, equal as represented, two closures $\langle f; \rho \rangle$, $\langle f; \rho' \rangle$ being strictly equal when ρ and ρ' are; or if they are eliminations of the same definition f with convertible arguments and spines. The common head is not unfolded; the arguments are compared by conversion, from stage 1, and this may unfold definitions inside them: comparing $f(g(a))$ with $f(b)$ may open g, never f.

2. *By head steps.* Otherwise the β-redexes at the head are reduced, without unfolding. If a definition blocks the head of one side only, that side is brought to weak head normal form. If both heads are blocked by definitions, the one declared later is opened by one δ-step. After each step, strict equality is tried.

3. *By components.* When no head is blocked, the outer forms are compared and their components are compared recursively, each from stage 1.

Stage 1 is a reader who recognises the same lemma at the same arguments; stage 2 is opening a definition, the δ-step of $\Lambda\Delta$. Stage 1 is essential: comparing arguments only strictly, and otherwise unfolding, takes more than 30 s on each file of the model tried, against 1.5–4 s. The strategy is uniform: there is no other choice of what to open, and no heuristic depending on the definitions. Sequential scope ensures that an earlier declaration's body cannot refer to a later declaration. It does not impose this order on instantiated values: if $f(x) := x$ is declared before $g := a$, then $f(g)$ unfolds to g. The later-first rule is a fixed policy, not a completeness argument. Two heads can also reduce to a common expression. If $h := a$ and then $k := a$, comparison proceeds as

$$h \sim k \quad \longmapsto \quad h \sim a \quad \longmapsto \quad a \sim a.$$

Accordingly, two `split` values, like two data types, are compared by name: by the identity of their code and their convertible environment arguments; their branches and motives are never compared; two record types are equal when they come from the same declaration with equal parameters, and their fields are never compared.

The expected type is opened only for introductions. An elimination (an application, a projection) is checked by inferring its type and comparing it with the expected type as it is;

the expected type is unfolded only when the term is an introduction (a λ, a pair) and its outer form is needed. A reader does not open the statement of a lemma to check that the lemma has been applied.

Notation. Omitted arguments are filled by one-way matching of the inferred types of the explicit arguments against the domains, never by unification; two different solutions for a hole are an error. A solution is written back as a term in the current scope. The names in scope are recognised by a restriction of stage 1 that opens nothing at any depth: strict equality, or the same definition at arguments compared in the same way, with no head step. A reader who writes an omitted argument recognises names and does not compute. Only when no name in scope denotes a definition occurring in the solution is that definition replaced by its body, which is then written. Equational chains `calc`, `open` and `import` are translated before checking.

3 The method

The measurement. With the option `--opened`, UNIT reports for each declaration the number of comparisons (including subproblems) settled at stage 1, by arguments (stage 1 with a recursive comparison of arguments), and by components (stage 3), and the definitions unfolded during its check, with multiplicities. We distinguish the *unfoldings*, each δ-step being one, from the distinct definitions opened: `opened 2:  Frege.her*3 Frege.Rplus*2` is five unfoldings of two definitions. The unfolding of a data type to its constructors is not counted: it is reading a declaration, not opening a construction. The counters are active while a declaration's checking result is evaluated, including argument matching and writing back inferred arguments when these are forced in that interval. They are not restricted to the final comparison of the inferred and expected types. Parsing and name resolution are outside this interval; diagnostic printing after the result is obtained is also outside it. With lazy evaluation, shared work is attributed to the declaration that forces it. The reported times are CPU times for these intervals. Unfolding counts describe this instrumented execution, not all operations of the checker and not a machine-independent cost of the proof.

What the text opens. In HOL every use of a definition is an explicit step, a rewrite with its defining theorem; in AUTOMATH and in UNIT the uses of a definition are silent. Objective type theory [4] does the same as HOL for computation rules: β and the computation of the identity eliminator hold only propositionally, with explicit witnesses, so each use is a transport written in the term. An informal text is in between: it says "by definition of X" where it uses a definition, and says nothing at steps it regards as obvious. Frege had no definitional equality for defining notions: in the *Begriffsschrift* [21] a definition is a judgement of identity of content, and each use of it is a step citing it. Each step of a text is classified by what a reader does:

step of the text	the reader	the checker should open
by Lemma L	matches names	nothing
by definition of X	opens X	X
without comment	nothing	nothing

With `--opens F`, where each line of `F` has the form `name:  `d_1 `...` d_k, UNIT prints for the declaration `name` the definitions it opened other than $d_1, \ldots, d_k$. Each is a use not listed in the supplied annotation. It may implement a representational convention or an ordinary silent use of a definition; its origin must be investigated before drawing a conclusion about the mathematical argument.

Reading a declaration. For a declaration that is slow, or opens many more definitions than its text cites:

1. write its informal proof;
2. say how a reader checks each step: by a lemma, by a definition, or without comment;
3. split the formal proof into these steps, one lemma for each step with its intermediate points written, and measure each;
4. for the step that costs more than its text, find the comparisons that cost, and the part of the checker that asks for them (a traced version prints each comparison costing more than a threshold with its caller: subsumption, a `refl`, the agreement of two solutions of a hole, the writing back of a solution, the arguments of the same definition, a `split`, a locked definition).

In these experiments the reports led to sharing one name for a construction, and to stating an interface or an intermediate identification once at the generic level. These are possible explanations and repairs, not an exhaustive classification of unexplained unfoldings.

Writing. The same method applies while writing, and gives rules: write the informal proof first; one name for each notion (no alias for something that has a name, pass a structure rather than its components, name the components of a construction once where it is defined); state the facts "by definition" once, by `refl`, next to the definition, and lock the construction after its characterising lemmas; write the intermediate point of a step that the text proves by two definitions; give accessors for the parts of elements rather than chains of projections; after each declaration, compare what it opened with what its text cites.

4 First results

4.1 Small examples

Sharing and the lock. On

```
module Mon (A : U a) (e : A) (op : A -> A -> A) {
  twice (x : A) : A = op x x ; } ;
module NM = Mon[N, zero, plus] ;
```

the report is:

declaration	statement	opened
`twiceUnfold`	`NM.twice x = plus x x`	`Mon.twice`
`byName`	`Mon[N,zero,plus].twice x = NM.twice x`	nothing
`opaque Mon.twice`		
`viaLemma`	`NM.twice one = plus one one`, by `twiceUnfold`	nothing
`transparent Mon.twice`		
`compAgain`	`NM.twice one = suc one`, by `refl`	`Mon.twice`, `plus`, `one`

The same instance reached by two paths is recognised without opening `twice`; after the lock, the lemma `twiceUnfold` is used instead of the definition.

Frege's transitive closure. The files `examples/frege.seg` and `examples/frege.opens` of the repository follow the first lemmas of the *Begriffsschrift* [21] on the transitive closure of a relation, as in [15, §2.4.3]. The context, a type A and a relation R, is a segment:

```
module Frege (A : U a) (R : A -> A -> U a) {
  her : [b] (A -> U b) -> U (a \/ b) =
    <b> \ P -> (x : A) -> P x -> (y : A) -> R x y -> P y ;
  Rplus (x y : A) : U (a+) =
    (P : A -> U a) -> her @a P -> ((z : A) -> R x z -> P z) -> P y ;
  lem1 (x : A) : her @(a+) (Rplus x) =
    \ y h1 z h2 P h3 h4 -> h3 y (h1 P h3 h4) z h2 ;
  her-step (P : A -> U (a+)) (hP : her @(a+) P) (x : A) (px : P x)
           (y : A) (r : R x y) : P y = hP x px y r ;
  lem2 (x y z : A) (h1 : Rplus x y) (h2 : Rplus y z) : Rplus x z =
    \ P h3 h4 -> h2 P h3 (\ u r ->
      her-step (Rplus x) (lem1 x) y h1 u r P h3 h4) ;
  lem3 (x y z w : A) (h1 : Rplus x y) (h2 : Rplus y z) (h3 : Rplus z w)
    : Rplus x w = lem2 x z w (lem2 x y z h1 h2) h3 ;
  base (x y : A) (r : R x y) : Rplus x y = \ P h3 h4 -> h4 y r ; } ;
```

It is instantiated, `module FS = Frege[N, Succ]`, at the successor relation `Succ x y = Id N (suc x) y`, and two declarations prove that 3 and 2 are in the closure of 0: `zero-three` by `FS.lem3` and `FS.base`, and `zero-two` directly, as a function of `P`, `h3` and `h4` that applies `h3` and `h4` to the two steps. Each line of `frege.opens` lists what the text opens at a step, and `--opens` reports the rest:

declaration	the text opens	also opened
`lem1`	`her`, `Rplus`	nothing
`her-step`	`her`	nothing
`lem2`	`Rplus`	nothing
`lem3`	nothing	nothing
`base`	`Rplus`	nothing
`zero-three`	`Succ`	`one`, `two`, `three`
`zero-two`	`Rplus`, `her`, `Succ`	`one`, `two`

The definition of hereditary is stated once, as the rule `her-step`, so that transitivity does not cite it; `lem3` composes two instances of transitivity by names, as the text does; through the lemmas, `zero-three` opens nothing of `Frege`, while the direct proof `zero-two` opens the closure and heredity, where a text would say "by definition". The numerals opened are a representation convention: a reader takes 1 to be the successor of 0. An earlier version of the checker also reported `her` in `lem2`, asked for by no comparison. Step 4 of the method traced it to the checker: a variable bound by λ, here `h3 : her @a P`, is tested for having a record type, and the test put its type in weak head normal form. This is the third case of (H2), unnecessary work in the checker; the test is now not counted as an opening, which also removed 167 702 unfoldings from the model of Table 2. Finally, Frege's proof of transitivity takes for P the closure `Rplus x` itself, which is in `U (a+)` and not in `U a`; the formal proof keeps the given P and uses `lem1` at the instance (x, y, h_1), as the comment in the file says. This is a change of argument that no unfolding report can detect, made inspectable by the accompanying explanation.

4.2 The developments

The method was applied to five developments.

- A model of univalent type theory over a base category with an interval, with finite posets and simplicial sets as instances, and the localisation at a modality of Sattler (18 051 declarations).
- Synthetic algebraic geometry [29], up to Châtelet's theorem: a Severi–Brauer variety with a point is projective space, from the axioms and a left exact sheafification for the étale topology (15 750 declarations).
- Synthetic Stone duality [27]: Stone spaces and countably presented Boolean algebras up to its Theorem 4.18, and sequential colimits and first cohomology up to its Theorem 5.16 (3300 and 4453 declarations in its two parts, which share some files).
- Cohomology in synthetic Stone duality [28], on top of the previous one: the cohomology of compact Hausdorff spaces, up to the comparison of H^n with Čech cohomology as a natural isomorphism of groups (8515 declarations with the files it reuses).
- Commutative algebra: finite free resolutions [11], §§2–8 (McCoy's theorems, the Buchsbaum–Eisenbud structure theorem, Hilbert–Burch), also without bases; Koszul and true grade and Wiebe's theorem [12]; local-global principles in depth 2 and Sharpe's theorem [10]; the Dedekind–Mertens lemma, gcd domains and polynomial rings over them, and the example of the Zariski main theorem of [1], done by Čech cohomology; Jacobian regularity and the flatness of $A[X,Y][1/J]/(F,G)$ over any commutative ring. These share a library of polynomials, determinants and exterior algebra (6742 to 14 138 declarations in six parts).

In the first two, about half of the declarations open no definition at all (10 026 and 9057). The last two were written from the start by the rules of Section 3, comparing after each file what its declarations open with what their texts cite.

Table 1 gives the check of the five developments by the checker described here.

development	entry point	declarations	failed	time
model of univalent type theory	`core`	18 051	0	46.5 s
algebraic geometry, Châtelet	`sag/chatelet`	15 750	0	6.8 s
Stone duality	`sag/stone`	3 300	0	1.0 s
sequential colimits, H^1	`sag/stoneseq`	4 453	0	1.3 s
cohomology, Čech comparison	`sag/ccacoh`	8 515	0	3.9 s
finite free resolutions	`sag/cffr`	10 806	0	6.0 s
without bases	`sag/cffru`	14 138	0	8.3 s
grade, Wiebe's theorem	`sag/wiebe`	8 303	0	3.0 s
depth 2, Sharpe's theorem	`sag/depth2`	8 335	0	3.4 s
gcd domains, Zariski main theorem	`sag/acl-zmt`	7 167	0	5.8 s
Jacobian, flatness	`sag/etale-flat`	6 742	0	2.4 s

Table 1: The developments checked by the current checker: declarations (with the files they use), failures, and CPU time of the whole run, loading included; single runs under macOS on arm64, GHC 9.6.7, `-O1`.

4.3 Totals

Table 2 gives, for each development, what checking opens: the declarations that open no definition, the unfoldings in all, the declarations with more than 500 unfoldings, and the largest number of unfoldings in one declaration. Stone duality and its cohomology, written by the rules

development	declarations	open nothing	unfoldings	above 500	largest
model of univalent type theory	18 051	10 026	1 858 930	601	171 693
algebraic geometry, Châtelet	15 750	9 057	255 822	61	3 130
Stone duality	3 300	1 780	44 797	5	953
sequential colimits, H^1	4 453	2 435	57 141	5	953
cohomology, Čech comparison	8 515	4 874	103 291	6	953
finite free resolutions	10 806	6 088	150 226	16	1 700
without bases	14 138	7 820	211 111	24	1 987
grade, Wiebe's theorem	8 303	4 660	107 958	10	1 360
depth 2, Sharpe's theorem	8 335	4 779	107 769	10	1 360
gcd domains, Zariski main theorem	7 167	3 855	1 968 312	49	407 664
Jacobian, flatness	6 742	4 812	73 365	9	8 910

Table 2: Unfoldings in checking the developments of Table 1 (`--opened`), with the files they use; each δ-step counts once, so a definition opened n times counts n times.

of Section 3, have six declarations with more than 500 unfoldings: five on Boolean algebras, shared by all three, and one in the cohomology. The model was written before these rules. In most of its files what is opened is the decoding of element types: a text writes an element of an extension $\Gamma.A$ as a pair (γ, a) without comment, while the checker opens the code of the extension at each projection. Its largest declaration opens one definition 140 464 times; such declarations have not yet been read against their texts. The exterior algebra and determinants of the algebraic geometry (2090 declarations with the files they use) cause 29 976 unfoldings; the largest declaration causes 1360, almost all of them (1255) of two aliases, `Alg` and `Ext`, of notions that have names. In the commutative algebra, the largest numbers come from the example of the Zariski main theorem: its polynomial identities in five variables are checked by computing normal forms, and each opens the multiplication and addition of the normal forms some 10^5 times (407 664 for the largest). This is computation stated as such in the text, not a definition opened silently.

5 Alignment

That a formal proof is accepted does not establish that it is the proof of the text. This is the point of [5], who show that the formalisation accompanying an announced proof of blow-up for the Navier–Stokes equations does not follow the text (some lemmas are proved in a different, weaker form), and argue that a faithful translation of a mathematical text cannot in general be guaranteed by a computation.

The aim here is to make selected aspects of the correspondence inspectable: the named constructions, applications of lemmas and openings of definitions in a formal proof are compared with a detailed, precise informal proof. The following table describes the intended organisation, rather than a criterion deciding fidelity:

informal proof	formal proof
a notion, with its name	one definition, used under that name
a lemma, a step	a declaration
an intermediate claim	a point written in the chain
"by Lemma L"	an application of L, checked by names
"by definition of X"	an opening of X, declared
a fact stated with a definition	a lemma proved by `refl`

The checker exposes computational work whose mathematical justification or representational origin must be explained. An opening absent from the annotation can arise from a second

name for a notion, decoding of a representation, an unstated intermediate identification, or an ordinary use of a definition left silent by the text. Each requires examination; none by itself proves that the arguments differ. Conversely, agreement of unfolding reports cannot establish that the statements or arguments correspond. A human reader must still compare both. The report supplies local evidence for that comparison, maintained while the proof is written.

A formal proof is well written, in this sense, when its structure is that of a detailed informal proof, and its openings can be explained by that text and its representation conventions. The informal proof this asks for is longer than usual: it has the explicit connections between convertible notions, each a lemma proved by `refl`, checked once, and free to use.

The repository has such accompanying texts for parts of the larger developments. `doc/chatelet.pdf` gives Châtelet's theorem as checked in `sag/chatelet`, with the changes made to the source note and a correspondence with the formal declarations; `doc/horrocks-divisor.pdf`, `doc/horrocks-determinant.pdf` and `doc/horrocks-gluing.pdf` give three proofs of rank-one Horrocks, the first with a correspondence to `sag/horrocks`, the last organised as the formal proof suggested; and `doc/finite-projective-flat.pdf` gives a correspondence for the characterisation of finite projective modules as finitely presented flat modules.

6 Related work

Definitions in the calculus. De Bruijn's $\Lambda\Delta$ [16] makes definitions part of the typed λ-calculus, with a reduction of their own, and his *mathematical vernacular* [17] is a language meant to be close to the way mathematics is written. The block structure of Automath is that of Unit. The theories of Half [13] are lists of declarations; Wang [30] implements a type theory with definitions as closures, which Unit follows.

Controlling unfolding. Implementations of type theory decide what to unfold by heuristics: the kernel described in [20] unfolds first the definition of greater height, which is close to our rule of opening the later definition, and other systems use priorities set by the user. Opaque definitions, and the controlled unfolding of [24], make unfolding a matter of the theory rather than of the implementation. Here the strategy is kept uniform, and the resulting reports are used to compare unfolding with an accompanying text.

Computation as explicit terms. In objective type theory [4] the only judgemental equality is syntactic identity; the computation rules of Π and of the identity type are propositional equalities with explicit witnesses, and type checking is quadratic. Its rules are interpretable in Unit, each witness becoming `refl` (the unbased eliminator being defined from the based one), but not conversely: Unit has more equalities, hence fewer models, and does not inherit the quadratic bound. The two make opposite choices for the same information. Objective type theory records each use of a computation in the proof term; Unit leaves the term unchanged, lets it serve at a definitionally equal type, and reports which definitions the comparison needed.

Structured formal proofs. Mizar, Isar [32] and Lamport's structured proofs [26] aim at formal texts whose structure is that of an informal proof; this is the aim of Section 5, which adds a measurement of what each step opens. De Bruijn's estimate of the ratio between the length of a formal text and that of the informal one [19] compares sizes, not structures.

Interfaces and constructions. In the formalisation of schemes in Lean [7], the algebraic gluing theorem was first proved for particular constructions of the localisation of a ring. The scheme construction then produced iterated localisations $R[1/f][1/g]$, canonically isomorphic to $R[1/fg]$ but not equal to it, and the canonical identifications that a text makes silently

had to be supplied as comparison maps and commuting diagrams. The repair was to state the theorem for any algebra satisfying a localisation predicate, which the authors took to be three elementary conditions due to Strickland, equivalent to the universal property. This is the second case of (H2), a missing interface, and the repair is the usual mathematical one: construct a localisation once, prove its universal property, and state everything else for an arbitrary algebra with that property. The localisations of `sag/affine` and `sag/closed` are written in this way. A localisation is an algebra with the universal property; one construction, adjoining y with $fy = 1$, shows that it exists; Strickland's conditions are derived as lemmas (that every element is a fraction, and that the kernel of $A \to A[1/f]$ consists of the elements annihilated by a power of f); and the iterated case is the file `LocalizationOverlap`, which shows that the localisation at g of a localisation at f is a localisation at fg. Its 44 declarations open the universal property, the algebra maps and their own definitions, and no definition of the construction (0.16 s). Univalence, available here, does not remove this work: to identify two localisations one must still construct the comparison isomorphism.

Human checking. Ganesalingam and Gowers [22] build a prover whose output reads as a human proof; in it, expanding a definition is a move of its own, of low priority, taken when nothing else applies. Ganesalingam [23] studies the language of mathematical texts, including definitions. Empirical studies of how mathematicians check proofs [31, 25] find that experts check steps locally, each against its justification, and spend their time on the steps whose justification is implicit. We know of no study of when a human reader opens a definition; the measurement proposed here gives a prediction that such a study could test.

7 Limitations and open questions

The results are preliminary: the classification of the steps of a text is done by hand, and the declarations of the model that open the most have not yet been read against their texts.

We do not give a soundness or completeness proof of the conversion algorithm. Reference [3] supplies the universe discipline; definitions follow $\Lambda\Delta$. The labelled inductive types with primitive recursion, nominal records, fixed fields and module rules described here are additional rules, not all part of [3]. Their combined metatheory remains to be supplied. Other open points include η for records and the restrictions on local units (a local block cannot introduce a data type or a module).

The checker, the examples and the developments are available at `https://github.com/coquand/unit`. The implementation is about 5700 lines of Haskell:

part	lines
syntax, values, evaluation, conversion, checking of units and segments	1850
omitted arguments: matching and writing back	980
universe levels: the decision of constraints [2]	170
parser and resolver	1740
printer, with the read-back of values to terms	170
strict positivity	130
driver, checking server and reports	620

Besides the Haskell base library, the checker uses only the standard packages `containers` (finite maps and sets) and, for the checking server, `directory`; the parser is written by hand, without a parsing library. Each declaration has a budget of ten seconds; a checking server loads a development once and rechecks only the changed files and those depending on them.

8 Conclusion

Unit is a type checker of about 5700 lines for a dependent type theory with modules in the tradition of Automath, with one treatment of definitions and nothing guessed. It has checked a model of univalent type theory, synthetic algebraic geometry up to Châtelet's theorem, and synthetic Stone duality with the cohomology of compact Hausdorff spaces. A small, uniform system is therefore enough for substantial mathematics, and its uniformity has a use of its own: the work of checking can be observed and compared with the informal proof. The extra notions of current proof assistants should be justified against this baseline, not assumed, the more so as formal proofs come to be written with the help of AI.

The system could be made smaller still, as de Bruijn argued in his plea for weaker frameworks [18]: without data types and recursive functions, the natural numbers and other inductive notions being primitive notions and axioms declared in a context, as in Automath, and with a simpler system of universes. This has a cost: what a `split` now computes, such as `plus` n `zero` $= n$, would become a lemma, used by transport, as in objective type theory [4]. Where to draw this line is the question of this paper: how much computation a checker should perform silently.

So simplified, Unit would be close to Carneiro's MM0 [14], which also aims at a small, efficient verifier for a weak framework: the object logic is given by declared axioms, and definitions are unfolded by a convertibility judgement of the framework, separate from the assertions of the object logic. MM0 separates a readable specification of axioms, definitions and statements from a proof certificate, produced by the MM1 front end, whose conversion steps, congruence and unfolding included, are explicit and can be shared. MM0 therefore already makes unfolding explicit and inspectable. The difference is where the unfolding steps are supplied: MM0 checks those written in a certificate; Unit chooses them while checking a dependent proof term, by its uniform strategy, and reports them, so that the work on the written proof can be related to the reading of an accompanying text.

Acknowledgements

The author supplied the design of Unit and directed its development and the interpretation of the checking experiments. Claude (Anthropic) produced the implementation code, and Claude and Codex (OpenAI) produced the formalisation code through iterative work under that direction. This attribution concerns code production; the mathematical sources are cited in the text. The resulting text and software remain the author's responsibility.

References

[1] M. E. Alonso, T. Coquand and H. Lombardi. Revisiting Zariski main theorem from a constructive point of view. *Journal of Algebra* 406 (2014), 46–68.

[2] M. Bezem and T. Coquand. Loop-checking and the uniform word problem for join-semilattices with an inflationary endomorphism. *Theoretical Computer Science* 913, 2022.

[3] M. Bezem, T. Coquand, P. Dybjer and M. Escardó. Type theory with explicit universe polymorphism. arXiv:2212.03284.

[4] B. van den Berg and M. den Besten. Quadratic type checking for objective type theory. arXiv:2102.00905, 2021.

[5] A. Bastounis, F. Circelli and A. C. Hansen. Navier–Stokes lost in translation: why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. arXiv:2610.08144, 2026.

[6] R. M. Burstall. Inductively defined functions in functional programming languages. *Journal of Computer and System Sciences* 34(2/3):409–421, 1987.

[7] K. Buzzard, C. Hughes, K. Lau, A. Livingston, R. Fernández Mir and S. Morrison. Schemes in Lean. *Experimental Mathematics* 31(2):355–363, 2022. arXiv:2101.02602.

[8] T. Coquand. An algorithm for type-checking dependent types. *Science of Computer Programming* 26(1–3):167–177, 1996.

[9] C. Cohen, T. Coquand, S. Huber and A. Mörtberg. Cubical type theory: a constructive interpretation of the univalence axiom. In *TYPES 2015*, LIPIcs 69, 2018. Implementation `cubicaltt`: `https://github.com/mortberg/cubicaltt`.

[10] T. Coquand and H. Lombardi. *Résolutions libres finies. Méthodes constructives.* Manuscript, 2024.

[11] T. Coquand and C. Quitté. Constructive finite free resolutions. *Manuscripta Mathematica* 137 (2012), 331–345.

[12] T. Coquand and C. Tête. An elementary proof of Wiebe's theorem. *Journal of Algebra* 499 (2018), 103–110.

[13] J. Cederquist. *A Pointfree Approach to Constructive Analysis in Type Theory.* PhD thesis, Chalmers, 1997.

[14] M. Carneiro. Metamath Zero: designing a theorem prover prover. In *Intelligent Computer Mathematics (CICM 2020)*, LNCS 12236, Springer, 2020, 71–88. doi:10.1007/978-3-030-53518-6_5.

[15] T. Coquand. Some remarks about dependent type theory. In B. Meyer, editor, *The French School of Programming*, Springer, 2023, 175–202. doi:10.1007/978-3-031-34518-0_8.

[16] N. G. de Bruijn. Generalizing Automath by means of a lambda-typed lambda calculus. In *Mathematical Logic and Theoretical Computer Science*, Lecture Notes in Pure and Applied Mathematics 106, Marcel Dekker, 1987, 71–92. Reprinted in *Selected Papers on Automath*, North-Holland, 1994.

[17] N. G. de Bruijn. The mathematical vernacular, a language for mathematics with typed sets. In *Workshop on Programming Logic*, Marstrand, 1987. Reprinted in *Selected Papers on Automath*, North-Holland, 1994.

[18] N. G. de Bruijn. A plea for weaker frameworks. In G. Huet and G. Plotkin (eds.), *Logical Frameworks*, Cambridge University Press, 1991, 40–67.

[19] N. G. de Bruijn. A survey of the project Automath. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980, 579–606.

[20] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, LNCS 12699, Springer, 2021.

[21] G. Frege. *Begriffsschrift, eine der arithmetischen nachgebildete Formelsprache des reinen Denkens.* Nebert, Halle, 1879. English translation in J. van Heijenoort, editor, *From Frege to Gödel*, Harvard University Press, 1967.

[22] M. Ganesalingam and W. T. Gowers. A fully automatic theorem prover with human-style output. *Journal of Automated Reasoning* 58, 2017, 253–291.

[23] M. Ganesalingam. *The Language of Mathematics*. LNCS 7805, Springer, 2013.

[24] D. Gratzer, J. Sterling, C. Angiuli, T. Coquand and L. Birkedal. Controlling unfolding in type theory. arXiv:2210.05420, 2022.

[25] M. Inglis and L. Alcock. Expert and novice approaches to reading mathematical proofs. *Journal for Research in Mathematics Education* 43, 2012, 358–390.

[26] L. Lamport. How to write a proof. *American Mathematical Monthly* 102, 1995, 600–608.

[27] F. Cherubini, T. Coquand, F. Geerligs and H. Moeneclaey. A foundation for synthetic Stone duality. arXiv:2412.03203, 2024.

[28] F. Cherubini, T. Coquand, F. Geerligs and H. Moeneclaey. Cohomology in synthetic Stone duality. Manuscript, 2026.

[29] F. Cherubini, T. Coquand and M. Hutzler. A foundation for synthetic algebraic geometry. arXiv:2307.00073.

[30] Q. Wang. *A Haskell Implementation for a Dependent Type Theory with Definitions*. Master's thesis, Chalmers, 2021.

[31] K. Weber. How mathematicians determine if an argument is a valid proof. *Journal for Research in Mathematics Education* 39, 2008, 431–459.

[32] M. Wenzel. Isar: a generic interpretative approach to readable formal proof documents. In *TPHOLs'99*, LNCS 1690, Springer, 1999.